Facultad de Ingeniería

Aplicación de TADs

Integrantes del grupo: Leandro Mendieta, Alex Pedrón.

Fecha de entrega: 04/05/2026

Link de repositorio: https://github.com/Valentin074/AyED2026c1---Mendieta-Pedron.git

Introducción:

El presente Trabajo Práctico tuvo como objetivo aplicar los contenidos desarrollados durante la cursada vinculados a estructuras de datos, programación orientada a objetos, análisis de algoritmos y medición de rendimiento computacional.

A lo largo del trabajo se resolvieron tres problemas principales. En primer lugar, se implementó una Lista Doblemente Enlazada como Tipo Abstracto de Dato (TAD), respetando operaciones específicas y criterios de eficiencia temporal y espacial. En segundo lugar, se desarrolló la estructura necesaria para ejecutar el juego de cartas “Guerra”, reutilizando la lista doblemente enlazada como base para representar mazos de cartas. Finalmente, se implementaron distintos algoritmos de ordenamiento para comparar su comportamiento práctico y teórico mediante pruebas experimentales.

Este trabajo permitió integrar conceptos teóricos vistos en clase con su aplicación práctica en Python, fortaleciendo la comprensión sobre complejidad algorítmica, modularización y diseño de soluciones eficientes.

Desarrollo de los Problemas:

Problema N°1: Implementación de Lista Doble Enlazada

A. Análisis y planteo

Se solicitó implementar una estructura dinámica capaz de almacenar elementos comparables, permitiendo inserciones y extracciones en ambos extremos de forma eficiente.

La elección de una lista doblemente enlazada resulta adecuada porque cada nodo mantiene referencias tanto al nodo siguiente como al anterior, permitiendo:

- Inserción al inicio en O(1)
- Inserción al final en O(1)
- Extracción de extremos en O(1)
- Recorrido bidireccional
- Uso eficiente de memoria dinámica

A diferencia de una lista estática, esta estructura no requiere realizaciones continuas de memoria.

B. Implementación / Resolución

Se desarrollaron dos clases:

Clase Nodo representa cada elemento de la estructura, almacenando:

- dato
- referencia al nodo siguiente
- referencia al nodo anterior

Clase ListaDobleEnlazada contiene:

- cabeza
- cola
- tamaño

Métodos implementados:

- esta_vacia()
- agregar_al_inicio()
- agregar_al_final()
- insertar()
- extraer()
- copiar()
- invertir()
- concatenar()
- len()
- add()
- iter()

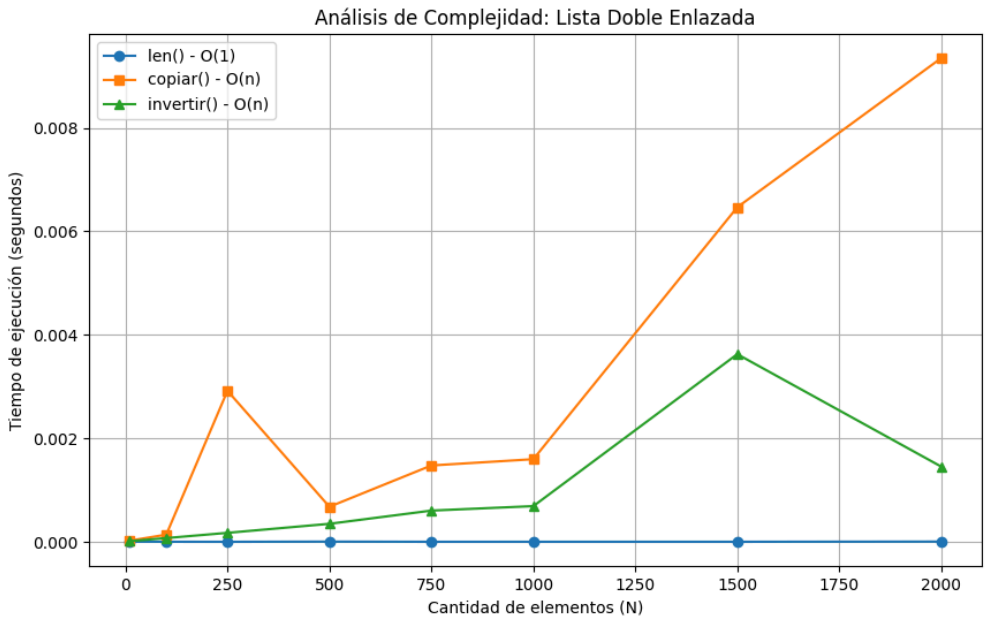
Se utilizaron propiedades (`@property`) para encapsular atributos privados.

C. Resultados y pruebas

La estructura permitió operar correctamente en todos los casos evaluados:

- Inserción en posiciones arbitrarias
- Extracción en extremos
- Copia independiente
- Inversión del orden
- Concatenación entre listas

A continuación, se presenta el análisis de rendimiento de la Lista Doble Enlazada mediante la medición de tiempos de ejecución:



La gráfica muestra que el método len() tarda lo mismo sin importar cuántos elementos haya, mientras que copiar() e invertir() tardan más a medida que la lista crece porque recorren cada nodo. Esto confirma que el código funciona de forma eficiente

Problema N°2: Juego de Cartas Guerra

A. Análisis y planteo

Se solicitó completar la implementación del juego Guerra mediante una clase Mazo, utilizando la lista doblemente enlazada desarrollada anteriormente.

El mazo debía comportarse como una cola doble:

- agregar cartas arriba
- agregar cartas abajo
- sacar cartas arriba

Esto convierte a la lista doblemente enlazada en una solución natural para el problema.

B. Implementación / Resolución

Se trabajó con tres módulos: **carta.py** define la clase Carta con:

- valor
- palo
- estado visible

También implementa comparación entre cartas según jerarquía:

2 < 3 < ... < 10 < J < Q < K < A

mazo.py Implementa:

- poner_carta_arriba()
- poner_carta_abajo()
- sacar_carta_arriba()
- **len**()
- **str**()

Internamente utiliza ListaDobleEnlazada.

juego_guerra.py Controla la lógica general:

- armado de mazo inicial
- reparto de cartas
- turnos
- guerras por empate
- detección de ganador o empate

C. Resultados y pruebas

El juego logró ejecutarse correctamente, simulando partidas completas entre dos jugadores.

Se verificó:

- reparto correcto de 52 cartas
- extracción por turnos
- guerras sucesivas
- acumulación de cartas ganadas
- finalización válida

También se controlaron excepciones cuando un mazo queda vacío.

Problema N°3: Algoritmos de Ordenamiento

A. Análisis y planteo

Se implementaron distintos algoritmos para comparar eficiencia práctica y teórica:

- Burbuja
- Quicksort
- Radix Sort
- sorted() de Python

Se generaron listas aleatorias de números de cinco dígitos con tamaños crecientes entre 1 y 1000 elementos.

B. Implementación / Resolución

Se midieron tiempos de ejecución utilizando el módulo. time, repitiendo el proceso para cada algoritmo y graficando resultados mediante matplotlib

C. Resultados y pruebas:

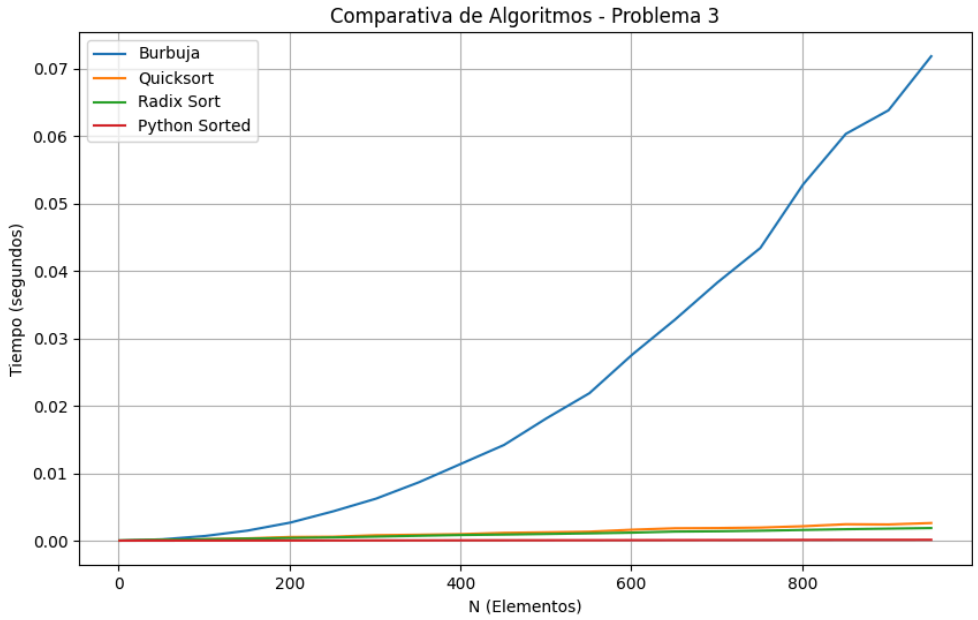
Observaciones:

- Burbuja fue el más lento al crecer N.
- Quicksort mostró muy buen rendimiento.
- Radix fue muy eficiente para enteros
- sorted() resultó el más optimizado por implementación interna.

¿Cómo funciona sorted()?

Python utiliza **Timsort**, algoritmo híbrido derivado de Merge Sort e Insertion Sort. Está optimizado para datos reales parcialmente ordenados.

A continuación, se muestra la gráfica comparativa de los tiempos de ejecución:



La gráfica muestra que el método Burbuja es el más lento, ya que su tiempo de ejecución se dispara rápidamente a medida que aumenta la cantidad de elementos. Por el contrario, Quicksort y Radix Sort mantienen tiempos mucho más bajos y estables, demostrando ser opciones mucho más eficientes para manejar listas grandes. Finalmente, la función sorted() de Python resulta ser la más veloz de todas.

Análisis de Resultados / Discusión

Los resultados prácticos confirmaron la teoría vista en clase.

Las estructuras dinámicas permiten mayor flexibilidad que arreglos estáticos, mientras que la elección del algoritmo de ordenamiento impacta fuertemente en el rendimiento.

Se observó que algoritmos simples como Burbuja sirven con fines didácticos, pero no son recomendables para grandes volúmenes de datos.

Asimismo, reutilizar la Lista Doble Enlazada en el juego Guerra permitió comprobar la importancia de diseñar componentes reutilizables.

Declaración de Uso Ético de Inteligencia Artificial

Herramientas utilizadas:

- Gemini
- Chat gpt

Usamos estas herramientas principalmente para sacarnos dudas teóricas sobre la lógica de los nodos en las listas dobles y para encontrar errores en el código que no estábamos viendo. También nos sirvieron de apoyo para organizar mejor las ideas y que la redacción del informe sea más clara.

Declaración de responsabilidad:

> "Los integrantes del grupo declaramos que hemos revisado, testeado y validado personalmente cada fragmento de código y explicación sugerida por las herramientas de IA. Entendemos la lógica detrás de cada estructura implementada y asumimos la responsabilidad total por la integridad técnica y académica de este trabajo. El resultado final refleja nuestro propio criterio y aprendizaje."

Trabajo en equipo

Para la organización del trabajo, dividimos las responsabilidades: Leandro Mendieta se encargó del desarrollo de los problemas 1 y 2, además de la generación de todas las gráficas de rendimiento; por su parte, Alex Pedrón llevó adelante la implementación del problema 3 y la redacción de gran parte de este informe. La comunicación fue constante a través de WhatsApp para coordinar dudas rápidas y utilizamos Google Docs como documento compartido para la puesta en común y el armado final del informe. Aunque no tuvimos inconvenientes graves en la resolución, se nos presentaron dificultades técnicas para integrar las distintas versiones del código, lo cual pudimos resolver más adelante.

Al iniciar el proyecto, estimamos que completaríamos el trabajo en un plazo de una semana. Sin embargo el tiempo real dedicado fue mayor al previsto. La principal causa fue la curva de aprendizaje inicial con las herramientas de desarrollo, especialmente en el problema 1, ya que no estábamos familiarizados con el entorno de Visual Studio Code. Una vez superada esta etapa y lograda una mejor configuración del entorno, el flujo de trabajo se volvió mucho más eficiente para la resolución de los problemas restantes.

Conclusiones

Se cumplieron todos los objetivos del trabajo y los códigos pasaron las pruebas de la cátedra. Con las gráficas vimos claramente la diferencia de velocidad entre los algoritmos y por qué conviene usar los más optimizados. También nos sirvió para aprender a organizar mejor el código en módulos y trabajar con documentos compartidos para el informe.

Bibliografía

- **Repositorio de Teoría de la Cátedra.** *Ejemplo de graficación y medición de tiempos*. https://github.com/FIUNER-LICA/ayed-codigos-teoria-para-estudiantes.
- **Plantilla de Práctica de la Cátedra.** *Estructura base y pautas para el desarrollo de trabajos prácticos*. https://github.com/FIUNER-LICA/ayed-repositorio-practica-plantilla.